

Comparação de Modelos Generativos para a Síntese de Imagens Artísticas

No âmbito da cadeira de Inteligência Artificial Generativa,
Departamento de Engenharia Informática, Universidade de Coimbra.

Bernardo Pedro (2021231014) e Íris Sousa (2021221971)

Abstract. Este trabalho aborda a geração automática de imagens artísticas através da comparação de diferentes modelos generativos aplicados ao dataset ArtBench-10. Foram testadas abordagens como VAE, CVAE, DCGAN, WGAN-GP e modelos de difusão, recorrendo às métricas FID e KID para avaliação. Os resultados mostram que os modelos de difusão apresentam o melhor desempenho, produzindo imagens mais realistas e consistentes. O treino com o conjunto completo de dados permitiu ainda melhorar a qualidade das amostras geradas, confirmando a eficácia desta abordagem para síntese de imagens artísticas.

Keywords: Machine Learning · Deep Learning · Generative Models · Conditional VAE (cVAE) · ArtBench10 · Image generation · Fréchet Inception Distance (FID) · Diffusion Models · GANs

1 Introdução

Este relatório tem como objetivo aplicar e comparar diferentes modelos generativos na síntese de imagens artísticas, utilizando o dataset ArtBench-10. Foram exploradas três principais abordagens: **Autoencoders Variacionais** (VAEs), **Redes Generativas Adversariais** (GANs) e **Modelos de Difusão**. O trabalho inclui o pré-processamento dos dados, a implementação e treino dos modelos, bem como a avaliação da qualidade das imagens geradas através de métricas como FID e KID.

O relatório encontra-se estruturado da seguinte forma: descrição do problema e do dataset, apresentação da metodologia e dos modelos implementados, análise dos resultados e, por fim, as conclusões.

2 Descrição do Problema e Dataset

O objetivo deste projeto é gerar imagens artísticas sintéticas que reproduzam a distribuição do dataset ArtBench-10, mantendo coerência visual e diversidade entre estilos. Foram comparadas diferentes arquiteturas generativas — VAEs, GANs e Modelos de Difusão — avaliando a sua capacidade de produzir imagens realistas e consistentes. O ArtBench-10 é composto por 60.000 imagens RGB de pinturas, distribuídas por 10 estilos artísticos distintos, com resolução de 32×32 pixels. Durante o desenvolvimento, foi utilizado um subconjunto de 20% dos dados para prototipagem, sendo posteriormente usado o conjunto completo para avaliação fina.

3 Metodologia

A nossa abordagem inclui o carregamento e pré-processamento dos dados, uma análise exploratória inicial, a implementação dos modelos, a avaliação da qualidade das imagens produzidas, e, por fim, a comparação dos diferentes modelos.

3.1 Pipeline de Dados

O dataset ArtBench-10 é carregado através da função `loadkaggleartbench10splits()`, que produz tensores RGB de 32×32 pixels com rótulos de 0 a 9 correspondentes às 10 classes artísticas. Para garantir reprodutibilidade, fixa-se a semente aleatória `SEED=42` em PyTorch, NumPy, random e CUDA.

O conjunto de treino contém 50.000 imagens (5.000 por classe) e o de teste 10.000 imagens (1.000 por classe), apresentando distribuição perfeitamente equilibrada, como visualizado na Fig. 1. Na Fase 1 (desenvolvimento), utiliza-se um subconjunto de 20% (10.000 imagens de treino); na Fase 2 (avaliação final), emprega-se o conjunto completo de treino (50.000 imagens).

O pré-processamento inclui conversão de $[0,255]$ para $[0,1]$ via `T.ToTensor()` seguida de `T.Normalize(mean=[0.5,0.5,0.5], std=[0.5,0.5,0.5])` para $[-1,1]$, aplicada em `HFDatasetTorch.__getitem__()`. Esta escala é compatível com ativações $\tanh$ dos modelos generativos e favorece gradientes estáveis nos VAEs. Os DataLoaders utilizam batch size de 128, shuffle=True (treino) e num_workers=4. Para FID/KID, as imagens são convertidas internamente pela `InceptionFeatureExtractor`. A visualização em `show_batch_grid()` aplica denormalização $\backslash(x = (x' + 1.0) / 2.0\backslash)$.



4 Modelos Implementados , Respetivas Funções e Métricas

Neste projeto foram implementados cinco modelos generativos — VAE, CVAE, DCGAN, WGAN-GP e DDPM. Todos utilizam imagens RGB de 32×32 normalizadas para o intervalo $[-1,1]$, com treino realizado utilizando o otimizador *Adam*, batch size de 128 e 70 épocas. A avaliação foi feita comparando 5.000 imagens geradas com 5.000 imagens reais, recorrendo às métricas FID e KID (Inception-v3).

4.1 Variational Autoencoder (VAE)

4.1.1 Arquitetura do modelo

Foi implementado um **Variational Autoencoder (VAE)** convolucional para a geração de imagens RGB de 32×32 . O modelo é composto por um encoder, que comprime a imagem num espaço latente, e um decoder, responsável pela sua reconstrução. O encoder produz os parâmetros μ e $\log(\sigma^2)$, que definem uma distribuição gaussiana no espaço latente. A amostragem é realizada através do reparameterization trick:

$$z = \mu + \sigma \cdot \epsilon, \text{ com } \epsilon \sim N(0, I).$$

O decoder reconstrói a imagem a partir de um vetor latente de dimensão 128, utilizando operações de upsampling e convolução. A ativação final $\tanh$ garante compatibilidade com a normalização dos dados.

Para melhorar a estabilidade do treino, foi utilizado *GroupNorm* no encoder, sendo mais adequado em cenários com batch sizes reduzidos.

4.1.2 Funções de perda

Durante o desenvolvimento, foram exploradas diferentes variantes da função de perda do VAE, composta por um termo de reconstrução e pela divergência KL, responsável por regularizar o espaço latente.

Numa fase inicial, foi utilizada a formulação combinando erro de reconstrução (L1 e MSE) com o termo KL. No entanto, esta abordagem revelou-se insuficiente na prática, apresentando resultados fracos (**FID** = **231.77**, **KID** = 0.1046 ± 0.0052), indicando dificuldade em capturar a distribuição dos dados.

Posteriormente, a função de perda foi refinada com a introdução de técnicas como *free bits* e *beta warm-up*, tornando o treino mais estável e melhorando a qualidade das amostras geradas. Estas alterações permitiram **reduzir significativamente o FID**, atingindo valores na ordem dos **154**.

```
def vae_loss(recon, x, mu, logvar, beta=0.5, recon_type='l1', free_bits=0.5):
    # Reconstruction
    if recon_type == 'l1':
        recon_loss = nn.functional.l1_loss(recon, x, reduction='mean')
    else:
        recon_loss = nn.functional.mse_loss(recon, x, reduction='mean')

    # KL with free bits
    kl = -0.5 * (1 + logvar - mu.pow(2) - logvar.exp())
    kl = torch.clamp(kl, min=free_bits)
    kl_loss = torch.mean(kl)

    return recon_loss + beta * kl_loss, recon_loss, kl_loss
```

4.1.3 Estratégia de treino

O treino foi realizado com recurso a *gradient clipping* e a um agendamento da taxa de aprendizagem do tipo *Cosine Annealing*, com o objetivo de melhorar a estabilidade e a convergência do modelo.

```
def linear_kl_beta(epoch, beta_start=0.0, beta_end=0.5, warmup_epochs=30):
    progress = min(1.0, epoch / float(warmup_epochs))
    return beta_start + (beta_end - beta_start) * progress
```

4.2 Conditional Variational Autoencoder (VAE)

Foi implementado um Conditional Variational Autoencoder (cVAE), que estende o VAE ao incorporar informação condicional associada aos estilos artísticos do dataset ArtBench-10. Ao contrário do VAE, que modela a distribuição global dos dados, o cVAE aprende a distribuição condicional $p(x|y)$, permitindo gerar imagens coerentes com um estilo específico.

A principal diferença arquitetural consiste na introdução dos rótulos tanto no encoder como no decoder. Os rótulos são convertidos em embeddings e concatenados com a entrada no encoder, e com o vetor latente no decoder, mantendo-se o restante da arquitetura inalterado.

A função de perda segue a mesma formulação do VAE.

4.3 Deep Convolutional GAN (DCGAN)

4.3.1 Arquitetura do modelo

O DCGAN é composto por duas redes: um *generator*, responsável por gerar imagens a partir de ruído, e um *discriminator*, que distingue entre imagens reais e geradas. O *generator* recebe um vetor latente $z \sim N(0, I)$ de dimensão 128 e projeta-o para uma representação de baixa resolução, que é progressivamente expandida até uma imagem 32×32 através de camadas de convolução transposta (*ConvTranspose2d*), com *BatchNorm* e ativações *ReLU*. A ativação final *tanh* é consistente com a normalização dos dados. O *discriminator* realiza o processo inverso, reduzindo progressivamente a dimensão espacial da imagem através de convoluções (*Conv2d*), utilizando ativações *LeakyReLU* e *BatchNorm*, até produzir um valor escalar.

A arquitetura segue as recomendações clássicas de DCGAN, utilizando apenas camadas convolucionais, sem fully connected intermédias, e substituindo operações de pooling por convoluções com stride.

4.3.2 Função de perda e treino

O treino segue a formulação adversarial clássica, com atualização alternada entre o *generator* e o *discriminator*. Ao contrário dos VAEs, não existe termo de reconstrução nem regularização explícita do espaço latente, sendo a aprendizagem guiada pelo feedback do *discriminator*. Esta abordagem permite gerar imagens mais nítidas, embora introduza maior instabilidade durante o treino.

4.4 Wasserstein GAN com Gradient Penalty (WGAN-GP)

4.4.1 Arquitetura do modelo

Para melhorar a estabilidade observada no DCGAN, foi implementado um WGAN-GP, que reformula a função de perda e introduz restrições adicionais ao modelo. O *generator* mantém uma arquitetura semelhante à do DCGAN, baseada na projeção do vetor latente seguida de upsampling progressivo com *ConvTranspose2d* e ativações *ReLU*, gerando imagens 32×32 com ativação final *tanh*. O *discriminator* é substituído por um *critic*, que atribui um valor contínuo às imagens em vez de uma probabilidade. Ao contrário do DCGAN, o *critic* não utiliza *BatchNorm*, de forma a garantir a restrição de Lipschitz.

4.4.2 Função de perda e estratégia de treino

A função de perda baseia-se na *Wasserstein distance*, permitindo obter gradientes mais estáveis durante o treino. Para garantir a **restrição de Lipschitz**, é introduzido um termo de *gradient penalty*, que penaliza desvios da norma do gradiente em relação a 1 em amostras interpoladas entre imagens reais e geradas. O treino inclui múltiplas atualizações do *critic* por cada atualização do *generator*, contribuindo para maior estabilidade e reduzindo problemas como *mode collapse*.

4.5 Diffusion

4.5.1 Arquitetura do Modelo

Foi implementado um modelo de difusão baseado em **DDPM** (*Denoising Diffusion Probabilistic Models*), com amostragem acelerada via DDIM. O modelo baseia-se em dois processos complementares: um processo *forward*, que adiciona ruído gaussiano à imagem ao longo de $T = 1000$ passos, e um processo *reverse*, em que uma rede neuronal aprende a remover esse ruído iterativamente, reconstruindo a imagem a partir de ruído puro. A rede responsável pela predição do ruído é uma **UNet** simétrica com blocos **ResNet** e *sinusoidal time embeddings*, que condicionam cada bloco ao passo temporal. As imagens geradas encontram-se no intervalo $[-1, 1]$, consistente com a normalização aplicada ao dataset.

4.5.2 Detalhes arquiteturais relevantes

A UNet implementada opera com $model_channels = 128$, resultando em 128 canais no encoder e 256 na fase intermédia. O *encoder* é composto por uma convolução inicial seguida de blocos ResNet com redução da resolução espacial através de convoluções com *stride*. A fase intermédia (*bottleneck*) aplica blocos ResNet adicionais sobre a representação comprimida, enquanto o decoder inverte o processo através de *upsampling* com *ConvTranspose2d* e *skip connections*, seguidos de blocos ResNet para refinamento. O condicionamento temporal é introduzido em cada bloco através de uma projeção do *time embedding*, com normalização *GroupNorm* e ativação SiLU.

Para melhorar a estabilidade, foi utilizada uma *Exponential Moving Average (EMA)* dos pesos do modelo (decay = 0.999), sendo estes utilizados na fase final de geração.

4.5.3 Processo de difusão e amostragem

O processo *forward* segue um *linear noise schedule*, com β a variar de 0.0001 até 0.02. A imagem ruidosa num passo t pode ser obtida diretamente a partir da imagem original, permitindo amostragem eficiente durante o treino. Na fase de geração, é utilizado o algoritmo DDIM com apenas 50 passos, reduzindo significativamente o custo computacional face aos 1000 passos do DDPM, sem comprometer de forma relevante a qualidade das amostras.

4.5.4 Função de perda

A função de perda é definida como o **erro quadrático médio (MSE)** entre o ruído real adicionado à imagem e o ruído predito pela rede. Esta formulação tem demonstrado empiricamente maior estabilidade e melhor qualidade de geração quando comparada com a otimização direta do ELBO.

4.5.5 Estratégia de treino

O modelo foi treinado durante 70 épocas utilizando o otimizador Adam, com taxa de aprendizagem inicial de 0.0002 e um agendamento do tipo *Cosine Annealing*. Durante o treino, os *timesteps* são amostrados aleatoriamente, garantindo que o modelo aprende a remover o ruído em diferentes níveis. Foram ainda aplicadas técnicas como *gradient clipping* e EMA dos pesos, contribuindo para maior estabilidade e consistência das amostras geradas.

```
def train_diffusion(model, train_loader, num_epochs, learning_rate, num_steps):
    """Train Diffusion model on dataloader. """
    model = model.to(DEVICE)
    optimizer = optim.Adam(model.parameters(), lr=learning_rate)

    # Cosine annealing decays LR smoothly to ~0 by the end of training
    scheduler = optim.lr_scheduler.CosineAnnealingLR(optimizer, T_max=num_epochs, eta_min=1e-6)
    schedule = DiffusionSchedule(num_steps=num_steps).to(DEVICE)
    ema = EMA(model, decay=0.999)
```

5 Resultados dos modelos (Subset 20% - Comparação)

5.1 Avaliação Quantitativa

Nesta secção apresentam-se os resultados da avaliação quantitativa dos modelos generativos. Foram utilizadas duas métricas padrão na literatura, o **FID** (*Fréchet Inception Distance*) e o **KID** (*Kernel Inception Distance*), que permitem avaliar a qualidade e a proximidade das imagens geradas relativamente às reais.

Para todos os modelos, foram geradas 5.000 imagens sintéticas e comparadas com 5.000 imagens reais do dataset ArtBench. O FID foi calculado sobre os conjuntos completos, enquanto o KID foi estimado a partir de 50 amostras de 100 imagens, sendo reportada a média e o respectivo desvio padrão.

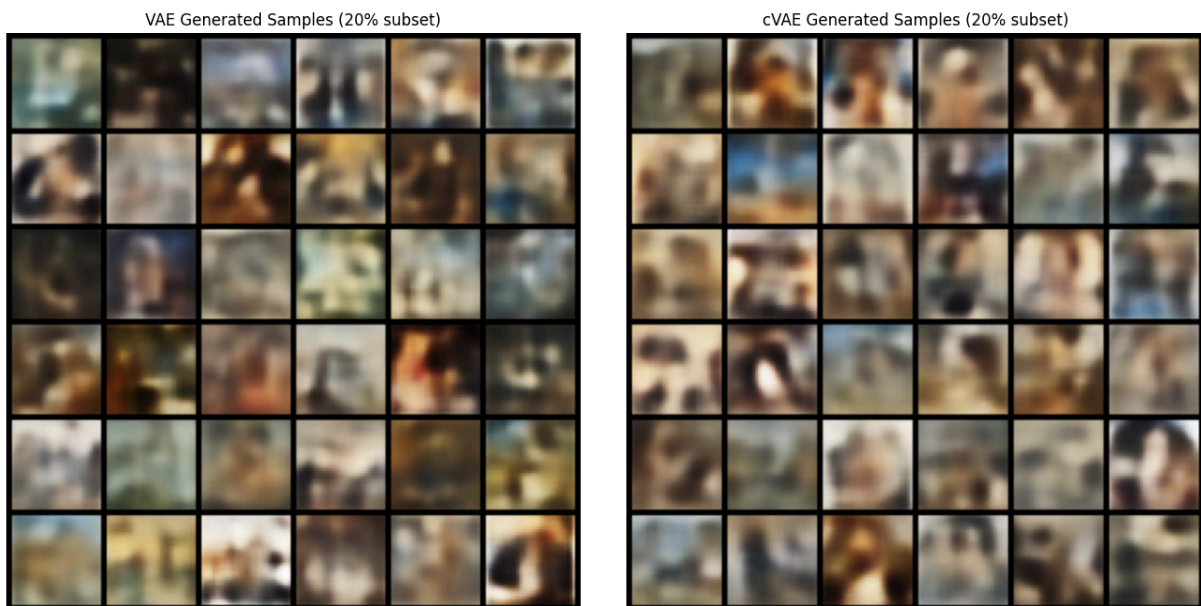
Resultados da Avaliação Quantitativa:

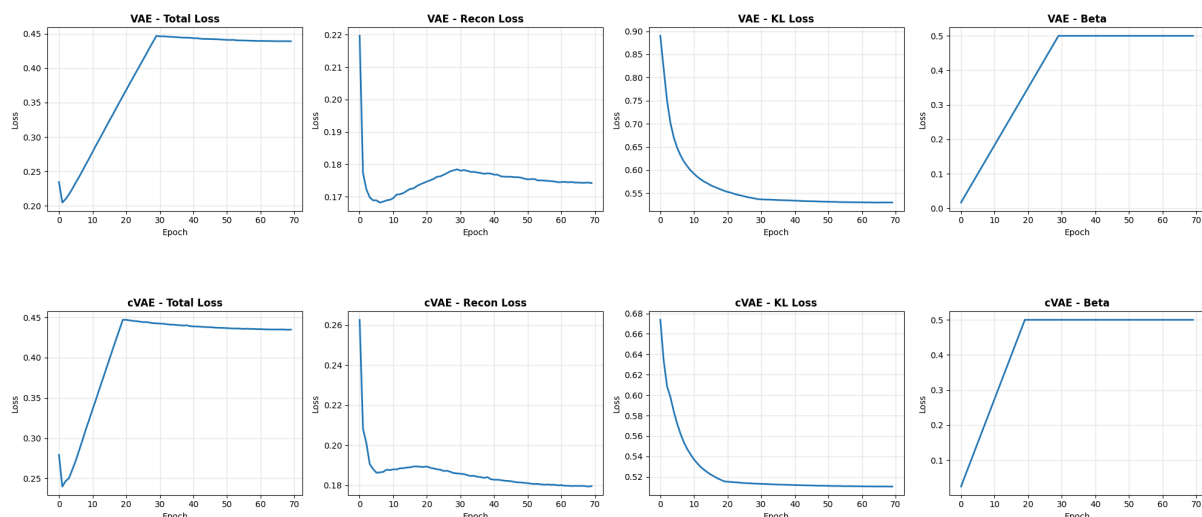
Modelo	FID ↓ (5000 vs 5000)	KID ↓ (média ± desvio padrão)
VAE	154.0133	0.064255 ± 0.003611
CVAE	165.7958	0.068678 ± 0.003097
DCGAN	80.5817	0.030754 ± 0.003324
WGAN-GP	119.1801	0.048638 ± 0.048636
Diffusion	40.3406	0.013406 ± 0.001754

Os resultados indicam que o **modelo Diffusion apresenta o melhor desempenho** em ambas as métricas, com $FID = 40.34$ e $KID = 0.01341 \pm 0.00175$, evidenciando uma melhor aproximação à distribuição das imagens reais. Os modelos GAN apresentam desempenho intermédio, com o DCGAN a superar o WGAN-GP, enquanto o VAE e o cVAE obtêm os piores resultados, associados à menor nitidez das amostras geradas. Embora, do ponto de vista teórico, o WGAN-GP e o cVAE sejam considerados mais robustos e expressivos do que o DCGAN e o VAE, respetivamente, tal vantagem não se refletiu neste contexto experimental. Este comportamento pode estar relacionado com fatores como a sensibilidade a hiperparâmetros, a estabilidade do treino e a convergência dos modelos. Com base nestes resultados, o modelo Diffusion foi selecionado para a Fase 2 do projeto.

5.2 Avaliação Qualitativa

5.2.1 Avaliação Qualitativa VAE vs CVAE

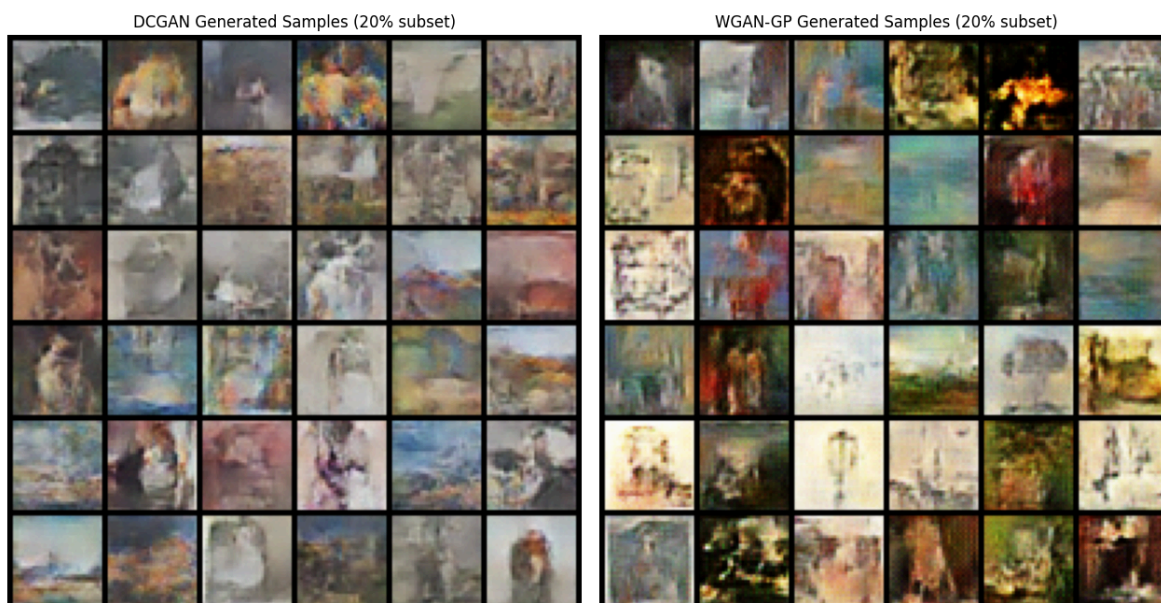


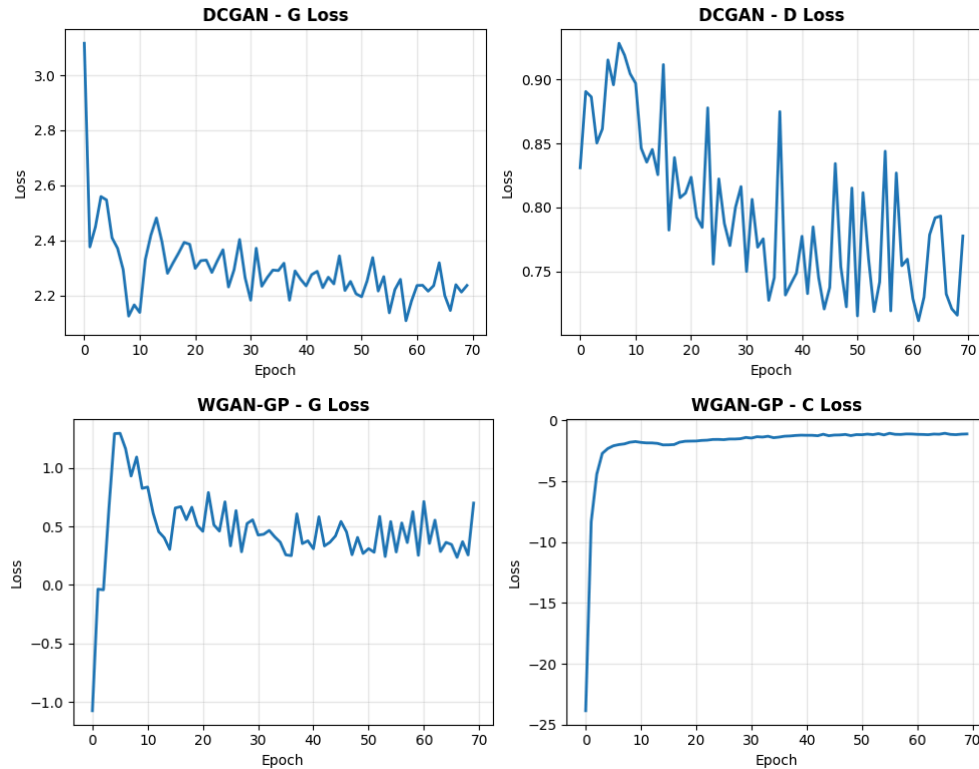


Os resultados não mostram diferenças claras entre o VAE e o CVAE, tanto a nível visual como quantitativo. À primeira vista, as imagens geradas pelos dois modelos são bastante semelhantes, sem uma melhoria evidente associada à introdução da condição no CVAE. Esta perceção é confirmada pelas curvas de loss, que seguem trajetórias muito próximas ao longo do treino. Tanto o erro de reconstrução como o termo de KL apresentam valores comparáveis, sem divergências consistentes entre os modelos.

No conjunto, isto sugere que, neste contexto experimental, a variável condicional não está a trazer um ganho efetivo no desempenho do modelo. Pode ser um sinal de que a informação condicional não é suficientemente informativa para os dados em questão, ou de que o modelo não a está a explorar de forma significativa.

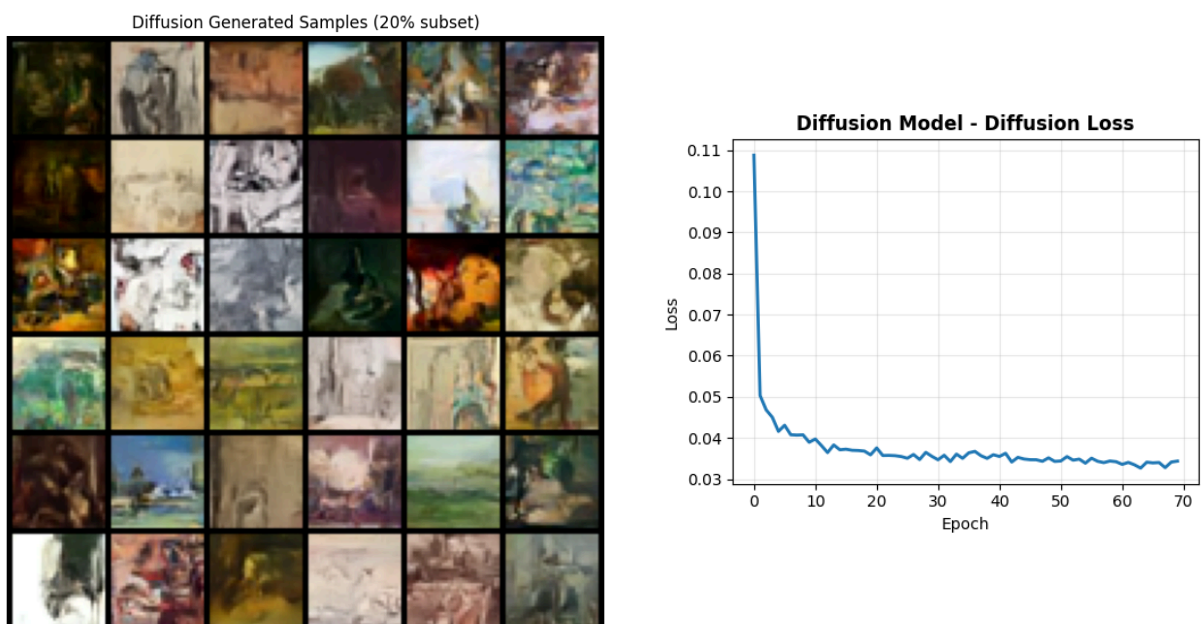
5.2.2 Avaliação Qualitativa DCGAN vs WGAN-GP





Embora o DCGAN apresente um FID mais baixo (89.44) do que o WGAN-GP (110.44), esta diferença não se traduz numa melhor qualidade percetual das imagens. A análise visual indica que o WGAN-GP gera amostras mais naturais, enquanto o DCGAN produz imagens com aspeto “esbranquiçado” e menor definição. Este comportamento é consistente com as *losses* observadas durante o treino. No DCGAN, as perdas do *generator* e do *discriminator* indicam um equilíbrio menos estável entre as redes. Já no WGAN-GP, apesar de as *losses* não serem diretamente comparáveis, observa-se um comportamento mais estável ao longo do treino. No conjunto, estes resultados sugerem que o FID, quando analisado isoladamente, pode não refletir totalmente a qualidade percetual das imagens, sendo o WGAN-GP globalmente mais consistente em termos visuais.

5.2.3 Avaliação Qualitativa Diffusion



As amostras geradas pelo modelo de difusão apresentam uma melhor coerência global, com estruturas facilmente reconhecíveis. Em comparação com os modelos GAN, observam-se transições mais suaves entre cores e formas, conferindo um aspeto mais natural, menos pixelizado e visualmente mais contínuo e “limpo”. Ainda assim, algumas amostras perdem alguma definição em detalhes mais finos, o que é expectável tendo em conta que o modelo foi treinado apenas com 20% do conjunto de dados. Apesar desta limitação, a qualidade visual global é mais estável e consistente do que nos modelos anteriores. Adicionalmente, verifica-se que a loss decresce rapidamente ao longo do treino, aproximando-se de valores próximos de zero, o que indica um processo de otimização estável e com erro global reduzido.

5.2.4 Análise Comparativa

De forma geral, o modelo de difusão apresenta o melhor desempenho entre os modelos testados. Comparativamente aos GANs, gera imagens com maior coerência global, transições mais suaves e menor presença de artefactos, resultando num aspeto mais natural e menos pixelizado. Apesar de alguma perda pontual de detalhe fino, a qualidade visual mantém-se mais estável e consistente. Adicionalmente, a loss apresenta uma descida rápida e tende para valores próximos de zero, indicando um processo de treino eficiente e bem convergente.

No conjunto, estes fatores mostram que o modelo de difusão foi o que alcançou melhores resultados globais em termos de qualidade de geração.

6 Melhor Modelo (Full Dataset 100%)

6.1 Avaliação Quantitativa (Final)

Após seleccionarmos o modelo de difusão como o melhor modelo, este foi re-treinado utilizando o conjunto completo de treino (100% dos dados), com o objetivo de avaliar o impacto da maior disponibilidade de dados no desempenho final. Os resultados reportados correspondem à **média** e ao **desvio padrão** obtidos ao longo de **múltiplas execuções com diferentes seeds**, tanto para o FID como para o KID.

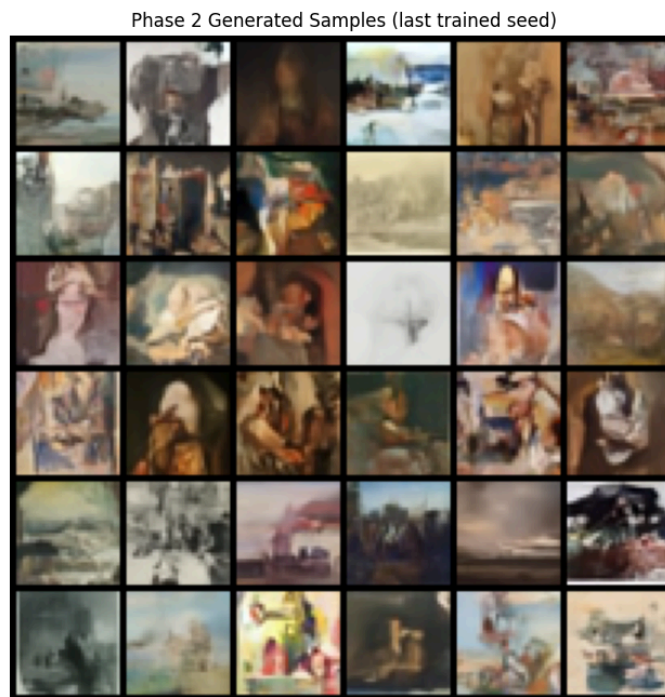
Os resultados obtidos encontram-se resumidos na seguinte tabela:

Seed	FID	KID (mean $\pm$ std)
0	29.5461	0.010971 $\pm$ 0.001596
1	29.6923	0.011261 $\pm$ 0.001792
2	29.8111	0.010878 $\pm$ 0.001704
3	29.5289	0.011222 $\pm$ 0.001262
4	29.7847	0.011708 $\pm$ 0.001878
5	29.6705	0.010782 $\pm$ 0.001839
6	29.8273	0.011075 $\pm$ 0.001507
7	29.5651	0.011016 $\pm$ 0.001637
8	29.9821	0.010810 $\pm$ 0.001435
9	30.0509	0.011567 $\pm$ 0.002014
Overall	29.7459 $\pm$ 0.1701	0.011129 $\pm$ 0.000297

Comparativamente com os resultados obtidos no subset de 20%, observa-se uma melhoria consistente nas métricas de avaliação após o aumento da quantidade de dados de treino. No caso do modelo treinado com 100% dos dados, verifica-se uma redução significativa do FID médio de **40.3370** para **29.7459 $\pm$ 0.1701**, indicando uma maior proximidade entre a distribuição das imagens geradas e a distribuição dos dados reais. De forma consistente, o KID também diminui de **0.013367 $\pm$ 0.001735** para **0.011129 $\pm$ 0.000297**, reforçando esta tendência de melhoria na qualidade das amostras geradas.

Este comportamento sugere que o **modelo de difusão beneficia claramente do aumento do volume de dados**, apresentando não só melhores métricas quantitativas, mas também uma maior estabilidade entre diferentes seeds, evidenciada pelo baixo desvio padrão observado.

6.2 Avaliação Qualitativa (Final)



As imagens geradas pelo modelo de difusão treinado com 100% dos dados mostram uma melhoria em relação à versão treinada com 20%. Há maior coerência visual, cores mais estáveis e menos artefactos, o que sugere uma aprendizagem mais completa da distribuição dos dados.

Ainda assim, continuam a existir algumas limitações, sobretudo na definição de detalhes finos e na consistência entre amostras. Isto indica que, apesar da melhoria obtida com mais dados, o modelo ainda não resolve totalmente os desafios associados à geração de imagens artísticas.

Em trabalho futuro, podia fazer sentido testar configurações mais robustas, aumentar o número de passos de amostragem, prolongar o treino e usar imagens de maior resolução, para perceber se o modelo consegue gerar imagens com melhor definição e manter mais consistência entre amostras.

7 Conclusão

Este projeto permitiu explorar e comparar diferentes modelos generativos aplicados à síntese de imagens artísticas, mostrando as diferenças entre abordagens baseadas em reconstrução, treino adversarial e difusão. Ao longo do trabalho, ficou claro que o desempenho dos modelos não depende apenas da arquitetura, mas também da estabilidade do treino, da quantidade de dados e da forma como cada modelo aprende a representar a distribuição das imagens.

De forma geral, os resultados mostram que o modelo de difusão foi o que teve melhor equilíbrio entre qualidade visual e métricas quantitativas, destacando-se face aos restantes. Ainda assim, foram identificadas algumas limitações, como a dificuldade em manter detalhes mais finos e a sensibilidade a certas configurações de treino, o que deixa espaço para melhorias futuras.

No geral, o trabalho ajudou a perceber, na prática, como diferentes modelos generativos se comportam num problema real de geração de imagens.

8 Referências

1. Kingma, D. P., & Welling, M. (2013). Auto-Encoding Variational Bayes. Disponível em: <https://arxiv.org/abs/1312.6114>
2. Sohn, K., Lee, H., & Yan, X. (2015). Learning Structured Output Representation using Deep Conditional Generative Models. NeurIPS. Disponível em: <https://proceedings.neurips.cc/paper/2015/hash/8d55a249e6baa5c06772297520da2051-Abstract.html>
3. Goodfellow, I., et al. (2014). Generative Adversarial Nets. NeurIPS. Disponível em: <https://arxiv.org/abs/1406.2661>
4. Arjovsky, M., Chintala, S., & Bottou, L. (2017). *Wasserstein GAN*. Disponível em: <https://arxiv.org/abs/1701.07875>
5. Ho, J., Jain, A., & Abbeel, P. (2020). *Denoising Diffusion Probabilistic Models*. NeurIPS. Disponível em: <https://arxiv.org/abs/2006.11239>
6. Song, J., Meng, C., & Ermon, S. (2020). *Denoising Diffusion Implicit Models*. Disponível em: <https://arxiv.org/abs/2010.02502>